Simple-Shell in C

Contributors:

- **Nihal, 2023345 (GoIDRoger69)**
- **Namit Bajaj, 2023340 (NAMIT-BAJAJ12)**

Github repository link :- [link to Repository]

Overview

This is a simple shell implementation in C. It can execute basic Unix commands, handle piping, execute commands in the background, and maintain a history of executed commands. The shell handles signal interruptions (e.g., **SIGINT** with **Ctrl+C**) and reaps child processes using **SIGCHLD**. Commands are executed in child processes via the **execvp()** function, while the shell itself provides an interactive environment for command execution.

Functionalities

1. Basic Command Execution

The shell can execute most basic Unix commands like ls, echo, grep, cat, wc, uniq, etc. It runs these commands using the **execvp()** function, creating a child process for each command.

2. Piping (|)

The shell supports piping of commands, where the output of one command is used as the input for another. This is handled by creating multiple child processes and piping their input and output streams.

3. Background Process Execution (&)

Commands ending with & are executed in the background. The shell does not wait for these commands to finish, and it immediately returns the prompt. The process ID **(PID)** of the background process is displayed.

4. Command History

The shell maintains a history of commands executed during the current session, including their process ID, start time, and duration (for foreground processes). You can view the history by typing **his**.

5. Signal Handling (SIGINT and SIGCHLD)

- **Ctrl+C (SIGINT)**: The shell intercepts Ctrl+C and prints a message instead of terminating the shell.
- **Child Process Reaping (SIGCHLD)**: Child processes are reaped to avoid zombies using the **SIGCHLD** handler.

6. Foreground vs Background Processes

- **Foreground Processes**: The shell waits for these processes to complete before returning control to the user. The execution time and details are stored in history.
- **Background Processes**: The shell immediately returns the prompt, and only the start time (not duration) is stored in the history.

Limitations

1. Complex Command-Line Arguments and Special Characters

The shell cannot handle commands that include quotes, escaped characters, or special characters, such as:

- ls some\ path

Reason: The command parsing uses **strtok()**, which splits the command on spaces, and does not account for quoted strings or escaped characters.

2. No Scripting Features

Features like conditionals (if, else), loops (for, while), and other scripting functionalities are not supported. The shell only executes individual commands provided by the user.

Reason: These are shell-internal features and would require additional logic for parsing and executing such constructs, which is beyond the scope of this simple shell.

3. No Built-in Shell Commands

Commands like cd, export, and unset are not supported because they are built-in commands specific to a shell's internal state, not external programs that can be executed using **execvp()**.

Reason: Shell-specific built-in commands like cd and export affect the shell process itself and require additional logic that is not covered by simply using **execvp()**.

4. Background Process Management

The shell does not currently offer a mechanism for monitoring or bringing background processes back to the foreground (e.g., using fg). You cannot terminate or check the status of background processes once they are running.

Reason: Implementing background process management would require additional features such as job control, which is outside the scope of this simple shell.

Contributions

- **Nihal**:
    - Added signal handling for **SIGCHLD**, ensuring that child processes are properly reaped to prevent zombie processes.
    - Designed and implemented the history feature, enabling the shell to keep track of executed commands, process IDs, start times, and execution durations.
    -
- **Namit**:
    - Implemented the piping functionality, allowing the shell to chain multiple commands using pipes **(|)**.
    - Implemented the background execution feature, allowing commands to run in the background with the **&** symbol.